

ORIENTACAO A OBJETOS

# CRUD com PHP + MySQL

## usando Classes e Objetos

Classe Conexao · Classe Equipamento · Classe EquipamentoDAO

**Conexao.php**

Gerencia a conexao PDO

**Equipamento.php**

Entidade / modelo de dados

**EquipamentoDAO.php**

Operacoes CRUD (DAO pattern)

# OOP — Classe Conexao | Conexao.php

```
<?php
// Conexao.php - Classe de conexao com PDO
class Conexao {
    private static $instancia = null;
    private $pdo; //extensão do PHP que permite acessar diferentes bancos de
    dados de forma padronizada e orientada a objetos

    private function __construct() {
        $dsn = 'mysql:host=localhost;dbname=industria;charset=utf8';
        //string de conexão que informa ao PDO onde e como conectar ao banco de dados.
        $this->pdo = new PDO($dsn, 'root', '', [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
        ]);
    }

    public static function getInstance(): PDO {
        if (self::$instancia === null) {
            self::$instancia = new Conexao();
        }
        return self::$instancia->pdo;
    }
}
?>
```

## Como funciona

### Singleton Pattern

Garante uma unica instancia de conexao em toda a aplicacao

### PDO::ERRMODE\_EXCEPTION

Lancara excecoes em caso de erro SQL — mais seguro que retornar false

### FETCH\_ASSOC

Retorna resultados como array associativo (chave = nome da coluna)

### getInstance()

Metodo estatico: chame Conexao::getInstance() em qualquer lugar do projeto

### \_\_construct() private

Impede criar new Conexao() diretamente — acesso so pelo getInstance()

# OOP — Classe Equipamento | Equipamento.php

```
<?php
// Equipamento.php - Classe de entidade (modelo)
class Equipamento {
    private int $id;
    private string $nome;
    private string $setor;
    private string $status;
    private string $dataCadastro;

    // Getters
    public function getId(): int { return $this->id; }
    public function getNome(): string { return $this->nome; }
    public function getSetor(): string { return $this->setor; }
    public function getStatus(): string { return $this->status; }

    // Setters
    public function setNome(string $v): void { $this->nome = $v; }
    public function setSetor(string $v): void { $this->setor = $v; }
    public function setStatus(string $v): void { $this->status = $v; }
}
?>
```

## Conceitos aplicados

### Encapsulamento

Atributos são private — acesso apenas pelos getters e setters públicos

### Type Hints (PHP 7+)

int, string, void garantem tipos corretos em tempo de execução

### Getters / Setters

Controlam leitura e escrita dos atributos — podem validar dados antes de salvar

### Responsabilidade Única

Esta classe só representa o equipamento, sem lógica de banco de dados

### Reutilização

O mesmo objeto é usado pelo DAO, pelo formulário e pela listagem PHP

# OOP — Classe EquipamentoDAO | EquipamentoDAO.php

```
<?php
// EquipamentoDAO.php - Operacoes CRUD via PDO
class EquipamentoDAO {
    private PDO $pdo;
    public function __construct() {
        $this->pdo = Conexao::getInstance();
    }
    // CREATE
    public function inserir(Equipamento $e): bool {
        $sql = 'INSERT INTO equipamentos (nome,setor,status) VALUES(?,?,?)';
        $st = $this->pdo->prepare($sql);
        return $st->execute([$e->getNome(),$e->getSetor(),$e->getStatus()]);
    }
    // READ
    public function listarTodos(): array {
        return $this->pdo->query('SELECT * FROM equipamentos')->fetchAll();
    }
    // UPDATE
    public function atualizar(Equipamento $e): bool {
        $sql = 'UPDATE equipamentos SET nome=?,setor=?,status=? WHERE id=?';
        $st = $this->pdo->prepare($sql);
        return $st->execute([$e->getNome(),$e->getSetor(),$e->getStatus(),$e->getId()]);
    }
    // DELETE
    public function excluir(int $id): bool {
        $st = $this->pdo->prepare('DELETE FROM equipamentos WHERE id=?');
        return $st->execute([$id]);
    }
}
?>
```

## Padroes usados

### DAO Pattern

Data Access Object: separa acesso a dados da logica de negocio e da interface

### Prepared Statements

prepare() + execute() previne SQL Injection — parametros nunca entram direto no SQL

### Injecao de Dependencia

O DAO recebe o PDO via Conexao::getInstance() — nao cria a conexao sozinho

### Tipagem forte

Equipamento \$e garante que apenas objetos validos entrem nos metodos

### Retorno bool

Os metodos retornam true/false para o controlador saber se a operacao foi bem-sucedida

# OOP — Usando as Classes nos Arquivos PHP

## salvar.php (CREATE)

```
// salvar.php — CREATE com OOP
require 'Conexao.php';
require 'Equipamento.php';
require 'EquipamentoDAO.php';

$e = new Equipamento();
$e->setNome($_POST['nome']);
$e->setSetor($_POST['setor']);
$e->setStatus($_POST['status']);

$dao = new EquipamentoDAO();
if ($dao->inserir($e)) {
    header('Location: listar.php');
} else {
```

## listar.php (READ)

```
// listar.php — READ com OOP
require 'Conexao.php';
require 'Equipamento.php';
require 'EquipamentoDAO.php';

$dao = new EquipamentoDAO();
$lista = $dao->listarTodos();

foreach ($lista as $row) {
    echo $row['id'] . ' - ';
    echo $row['nome'] . ' | ';
    echo $row['setor'];
    echo '<br>';
}
```

## Fluxo com OOP

### require das 3 classes

Sempre importe Conexao, Equipamento e EquipamentoDAO antes de usar

### new Equipamento()

Cria o objeto e popula com setters antes de passar ao DAO

### new EquipamentoDAO()

O DAO já obtém o PDO automaticamente via Conexao::getInstance()

### Separacao de papeis

PHP de formulario só conhece Equipamento; banco fica dentro do DAO

### Pronto para escalar

Para outros modelos (Funcionario, Maquina) basta criar novas classes